

Notes on the “pirate” dedisperser: tree dedispersion

Kendrick Smith

September 7, 2026

Contents

1	Tree dedispersion	2
1.1	Tree gridding kernel	2
1.2	Tree dedispersion	3
1.3	Subband search	4
1.3.1	Parameterization of subbands	4
1.3.2	Subbanded dedispersion	5
1.4	Peak-finding	7
1.4.1	Peak-finding kernels	8
1.5	Time-downsampled trees and early triggers	8
1.6	Dedispersion output arrays	12
1.6.1	Parameter definitions	12
1.6.2	Output array shapes	12
1.6.3	The out_argmax array	13
2	Implementation	14
2.1	Non-recursive formulation of tree dedispersion	14
2.2	The splitting identity	14
2.3	GPU implementation	15
2.4	Extracting subbands from a split dedispersion [Claude]	17
2.5	Alignment constraints and the MegaRingbuf	17
2.6	Downsampled trees and the LaggedDownsampler	18
2.7	Putting it all together: the GPU kernel diagram	20
A	Derivations [Claude]	22
A.1	The non-recursive formulation of tree dedispersion	22
A.2	The splitting identity	23
A.3	The upper-half identity	24

1 Tree dedispersion

1.1 Tree gridding kernel

Our starting point for tree dedispersion is a shape $(N_{\text{freq}}, N_{\text{time}})$ array representing channelized intensity time series. We do not assume that frequency channels are evenly spaced (in CHORD we use unequal spacing). We want to search this array for FRBs with delay $\propto \nu^{-2}$.

The first step is a “tree gridding” kernel which changes variables so that the delay is linear (i.e. the FRB dispersion sweep looks like a straight line). The input of the tree gridding kernel is a shape $(N_{\text{freq}}, N_{\text{time}})$ array, and the output is a shape $(N_{\text{tree}}, N_{\text{time}})$ array, where $N_{\text{tree}} = 2^r$. The parameter r is the tree dedispersion “rank” and is specified in a config file.

The leading axis of the input array represents frequency channels, and will be indexed by $0 \leq F < N_{\text{freq}}$. The leading axis of the output array represents a reparameterized frequency channel (“tree-freq” channel for short), and will be indexed by $0 \leq f < 2^r$. Conceptually, the tree gridding kernel just reparameterizes the frequency axis (i.e. the time axis is a spectator.)

The tree gridding kernel also takes an auxiliary array `channel_map`, which describes the mapping between frequency channels $0 \leq F < N_{\text{freq}}$ and tree-freq channels $0 \leq f < N_{\text{tree}}$. The array `channel_map` has length $N_{\text{tree}} + 1$, and its entries are (in general non-integer) frequency-channel indices in the range $[0, N_{\text{freq}}]$. Writing c_f for the f -th entry, tree-freq channel f is defined to occupy the half-open interval of frequency channels

$$I_f = [c_{f+1}, c_f), \quad 0 \leq f < N_{\text{tree}}, \quad (1.1)$$

i.e. consecutive entries give the lower and upper edges of each tree-freq channel. The entries are fixed by the dispersion geometry: the tree-freq channels are equally spaced in the dispersion variable $x \equiv \nu^{-2}$ (this is the change of variables that linearizes the delay), so the f -th edge sits at

$$x_f = \nu_{\text{hi}}^{-2} + \frac{f}{N_{\text{tree}}} (\nu_{\text{lo}}^{-2} - \nu_{\text{hi}}^{-2}), \quad (1.2)$$

and c_f is the frequency-channel index of the frequency $\nu_f = x_f^{-1/2}$, where the index increases with frequency, running from 0 at the band bottom ν_{lo} to N_{freq} at the band top ν_{hi} . In particular $c_0 = N_{\text{freq}}$ and $c_{N_{\text{tree}}} = 0$, so `channel_map` is monotonically *decreasing* ($c_{f+1} < c_f$). We store it in double precision, since the gridding weights below are obtained by differencing adjacent entries, which loses relative precision.

The details of the tree gridding kernel are as follows. Each output tree-freq channel f is a weighted sum of the input frequency channels whose bins overlap the interval $I_f = [c_{f+1}, c_f)$:

$$\text{out}[f] = \sum_{F=0}^{N_{\text{freq}}-1} w_{fF} \text{in}[F], \quad (1.3)$$

where $\text{in}[F]$ and $\text{out}[f]$ are length- N_{time} time series, and the same weights are applied at every time sample (the time axis is untouched). The weight w_{fF} is the length of the overlap between the tree-freq channel interval I_f and the unit-width frequency bin $[F, F+1)$:

$$w_{fF} = \max(0, \min(c_f, F+1) - \max(c_{f+1}, F)). \quad (1.4)$$

A frequency channel lying entirely within I_f contributes with weight 1, while a channel straddling an edge of I_f contributes a fractional weight. Because the intervals I_f tile $[0, N_{\text{freq}}]$, the weights of any fixed input channel sum to one, $\sum_f w_{fF} = 1$; the gridding kernel is thus a delay-linearizing rebinning of the frequency axis that conserves total intensity, $\sum_f \text{out}[f] = \sum_F \text{in}[F]$.

Note that the tree gridding kernel is a monotone *decreasing* reindexing operation. Thus, to search for FRBs with positive dispersion (lowest frequencies arise last), we want to search over lines in the tree array

with positive slope (highest tree-freq channels arrive last).

1.2 Tree dedispersion

After the tree gridding kernel, the next step is “tree dedispersion”, which searches for straight lines with $0 \leq \text{slope} \leq 1$. (The extension to slope > 1 will be discussed in a future section, along with more details such as subband searches, early triggers, and peak-finding.)

The dedispersion input array has shape $(2^r, N_{\text{time}})$, where the leading axis is “tree-freq” (see above). Straight lines will be parameterized by an integer $0 \leq d < 2^r$, representing the delay (in time samples) between the lowest and highest tree-freq channel, and a time index t representing the arrival time in the *highest* tree-freq channel (i.e. the *latest* arrival time). Thus, the dedispersion output array also has shape $(2^r, N_{\text{time}})$, where the first axis represents a delay (or DM).¹

Tree dedispersion is a composition of functions:

$$(\text{Input array}) \xrightarrow{\text{DDS}(0)} (\text{Intermediate array}) \xrightarrow{\text{DDS}(1)} \dots \xrightarrow{\text{DDS}(r-1)} (\text{Output array}) \quad (1.5)$$

where the function $\text{DDS}(k)$ (“dedispersion step k ”) is described as follows:

- The $\text{DDS}(k)$ input array is a shape $(2^{r-k}, 2^k, N_{\text{time}})$ array. The first axis represents a coarse frequency channel. Each coarse-freq channel f corresponds to a block of 2^k tree-freq channels in the original tree input array:

$$2^k f \leq (\text{tree-freq index}) < 2^k(f+1) \quad (1.6)$$

The second axis represents a delay $0 \leq d < 2^k$ within the coarse-freq channel (1.6). As k increases, the coarse-freq channels double in width, and we need twice as many delay indices to cover the range $0 \leq (\text{slope}) \leq 1$.

The time index represents an arrival time at the largest tree-freq channel in the coarse-freq channel (1.6), i.e. the latest arrival time within the coarse-freq channel.

- The $\text{DDS}(k)$ operation is defined by:

$$\begin{aligned} \text{out}[f, 2d, t] &= \frac{1}{\sqrt{2}} (\text{in}[2f, d, t-d] + \text{in}[2f+1, d, t]) \\ \text{out}[f, 2d+1, t] &= \frac{1}{\sqrt{2}} (\text{in}[2f, d, t-d-1] + \text{in}[2f+1, d, t]) \end{aligned} \quad (1.7)$$

- As formulated above, tree dedispersion can lead to negative time indices. By default, $\text{in}[\dots, t]$ with $t < 0$ is defined to be zero. (In the real-time system, we run tree dedispersion in an “incremental” context where a negative time index should be interpreted as a reference to data received in a previous time chunk.)
- The $\text{DDS}(0)$ input array is sometimes reshaped from $(2^r, 1, N_{\text{time}})$ to $(2^r, N_{\text{time}})$, and the $\text{DDS}(r-1)$ output array is sometimes reshaped from $(1, 2^r, N_{\text{time}})$ to $(2^r, N_{\text{time}})$. Thus, we think of the entire tree dedispersion operation (1.5) as having input and output shapes $(2^r, N_{\text{time}})$.

We define $\text{DD}(r)$ (“rank- r dedispersion”) to be the composition of all r steps in Eq. (1.5):

$$\text{DD}(r) = \text{DDS}(r-1) \circ \dots \circ \text{DDS}(0) \quad (1.8)$$

¹Frequently in the code (but not in these notes), it’s convenient to use a *bit-reversed* delay index $0 \leq d_{\text{rev}} < 2^r$. (This turns out to allow some array operations to operate “in place”, facilitating parallelization.)

1.3 Subband search

So far, we have described a search for FRBs which cover the full tree-freq range $0 \leq f < 2^r$. This would be suboptimal since real FRBs do not always cover the full frequency range. Next, we introduce a “sub-band” search for searching a configurable set of subbands.

1.3.1 Parameterization of subbands

Our dedispersion config files contain a length- $(R + 1)$ integer-valued array of subband counts C_l .² Here $0 \leq R \leq r$, where r is the tree rank. The counts C_l parameterize a set of subbands as follows:

- Frequency subbands are indexed by a pair (l, s) where $0 \leq l \leq R$ and $0 \leq s < C_l$. We will sometimes “flatten” the pair (l, s) into a single subband index $0 \leq n < N$, where

$$N \equiv \sum_{l=0}^R C_l \quad (1.9)$$

is the total number of subbands. (In the code, n and N are denoted by the same symbols, e.g. N is `FrequencySubbands::N`.)

- Level $l = 0$ is special: subband $(0, s)$ corresponds to tree-freq range

$$2^{r-R}s \leq (\text{tree-freq index}) < 2^{r-R}(s+1) \quad (1.10)$$

- At levels $l > 0$, subband (l, s) corresponds to tree-freq range

$$2^{r-R+l-1}s \leq (\text{tree-freq index}) < 2^{r-R+l-1}(s+2) \quad (1.11)$$

Thus, level 0 consists of non-overlapping bands whose fractional width is $1/2^R$. Levels $l > 0$ consist of bands whose fractional width is $1/2^{R-l}$, and overlap by a factor 2.

- To avoid “overflowing” the band, C_l must satisfy

$$0 \leq C_l \leq \begin{cases} 2^R & \text{for } l = 0 \\ 2^{R-l+1} - 1 & \text{for } l > 0 \end{cases} \quad (1.12)$$

A level with $C_l = 0$ is allowed, and simply contributes no subbands. We require $C_R = 1$, i.e. we always search the full frequency band. The case $C_l = \{1\}$ (with $R = 0$) corresponds to disabling subbands, and only searching the full band.

- The details of this scheme, including the “specialness” of level $l = 0$, are dictated by convenience in the GPU kernel.
- To *interpret* the C_l already in a config file, run `pirate_frb show dedisperser -cv`. For example, for $C_l = [5, 9, 7, 3, 1]$ this prints:

```
# In this config, there are 25 top-level frequency subband(s):
#   pf_level=0: [948.7,1500.0], [750.0,948.7], [639.6,750.0], [566.9,639.6], [514.5,566.9]
#   pf_level=1: [750.0,1500.0], [639.6,948.7], [566.9,750.0], [514.5,639.6], [474.3,566.9],
#               [442.3,514.5], [416.0,474.3], [393.9,442.3], [375.0,416.0]
#   pf_level=2: [566.9,1500.0], [474.3,750.0], [416.0,566.9], [375.0,474.3], [344.1,416.0],
#               [319.8,375.0], [300.0,344.1]
#   pf_level=3: [416.0,1500.0], [344.1,566.9], [300.0,416.0]
```

²In the code, R is denoted `pf_rank`, l is denoted `pf_level`, and C_l is denoted `frequency_subband_counts`.

```
# pf_level=4: [300.0,1500.0]
```

- To *create* a C_l array with a given $\max \Delta(\log \nu)$, run `pirate_frb dev make_subbands`. For example, `pirate_frb dev make_subbands 300 1500 0.1 -r 4` gives $C_l = [5, 9, 7, 3, 1]$. These subbands are shown visually in Figure 1.

1.3.2 Subbanded dedispersion

Subbanded dedispersion is an operation whose input array is the same as ordinary (non-subbanded) tree dedispersion (1.2), i.e. a shape $(2^r, N_{\text{time}})$ array where the first index is tree-freq. The output array has shape $(2^{r-R}, M, N_{\text{time}})$, where M is the number of “multiplets”:

$$M \equiv \sum_{l=0}^R 2^l C_l \quad (1.13)$$

The code requires $R \leq 4$ (`constants::max_peak_finding_rank`), so saturating the bounds on C_l above gives $C_l = [16, 15, 7, 3, 1]$, and hence $N \leq 42$ and $M \leq 114$.

Each subband (l, s) “owns” a shape $(2^{r-R}, 2^l, N_{\text{time}})$ subarray of the output. The first two axes correspond to 2^{r-R+l} dispersion delays across the subband, split into a “coarse” index $0 \leq d_{\text{hi}} < 2^{r-R}$ and a “fine” index $0 \leq d_{\text{lo}} < 2^l$. (This index-splitting ensures that the combined output of all subbands is a regular $(2^{r-R}, M, N_{\text{time}})$ array, rather than a “ragged” object.)

Conceptually, subband dedispersion works as follows. For each subband (l, s) , we apply tree dedispersion to the appropriate tree-freq range (either Eq. (1.10) or (1.11)). We also apply a time lag so that the time index corresponds to the extrapolated arrival time at the highest tree-freq in the full band, not the highest tree-freq in the subband. For an efficient implementation, we run tree dedispersion as usual (Eq. 1.5), and extract per-subband dedispersion outputs from intermediate arrays “on the fly”, rather than dedispersing each subband “from scratch”.

Formally, the details are as follows:

- **Case 1: $l = 0$ or even s .** In this case, the subband corresponds to an “aligned” tree-freq range of the form:

$$2^{r-R+l} f \leq (\text{tree-freq index}) < 2^{r-R+l} (f + 1) \quad (\text{where } 0 \leq f < 2^{R-l}) \quad (1.14)$$

where the coarse-freq index f is given by:

$$f = \begin{cases} s & \text{if } l = 0 \\ s/2 & \text{if } l > 0 \end{cases} \quad (1.15)$$

The subband owns a shape $(2^{r-R}, 2^l, N_{\text{time}})$ subarray of the output (denote by `out`).

In this case, the per-subband dedispersion outputs already arise as intermediates in tree dedispersion (Eq. 1.5); we just need to apply a time lag. Let `tmp` be the output array of `DDS(k-1)`, where $k = r - R + l$. Then `tmp` has shape $(2^{R-l}, 2^{r-R+l}, N_{\text{time}})$. To populate the `out` array, we slice/reshape `tmp` and apply time lags:

$$\text{out}[d_{\text{hi}}, d_{\text{lo}}, t] = \text{tmp}[f, d, t - T] \quad (1.16)$$

where on the RHS we define:

$$d \equiv d_{\text{hi}} 2^l + d_{\text{lo}} \quad T \equiv d_{\text{hi}} 2^l (2^{R-l} - 1 - f) \quad (1.17)$$

- **Case 2: $l > 0$ and odd s .** In this case, the subband corresponds to a “half-aligned” tree-freq range

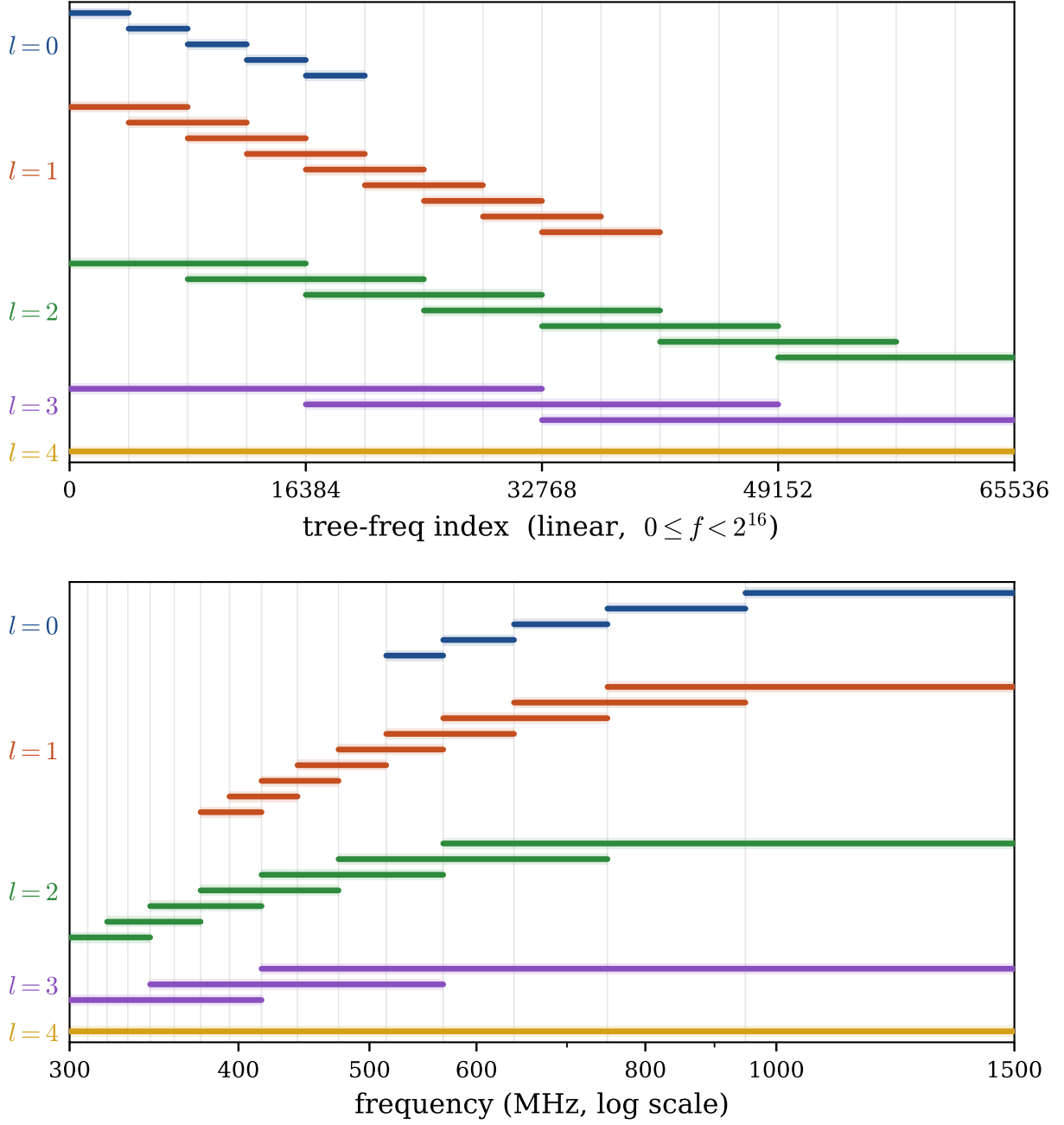


Figure 1: Frequency subbands in a CHORD config with `frequency_subband_counts=[5,9,7,3,1]`. The top panel shows the subbands in tree-freqs (where they are “naturally” defined) and the bottom panel shows the conversion to physical frequency ν in MHz. Note that the conversion between tree-freqs and ν is monotone decreasing.

of the form:

$$2^{r-R+l-1}(2f+1) \leq (\text{tree-freq index}) < 2^{r-R+l-1}(2f+3) \quad (\text{where } 0 \leq f < 2^{R-l} - 1) \quad (1.18)$$

where the coarse-freq index f is given by $f \equiv (s-1)/2$. The subband owns a shape $(2^{r-R}, 2^l, N_{\text{time}})$ subarray of the output (denote by **out**).

This case is a little more complicated, since the per-subband dedispersion outputs are “one dedispersion step away” from the intermediate arrays in tree dedispersion. Let **tmp** be the output array of $\text{DDS}(k-1)$, where $k = r - R + l - 1$. Then **tmp** has shape $(2^{R-l+1}, 2^{r-R+l-1}, N_{\text{time}})$.

$$\begin{aligned} \text{out}[d_{\text{hi}}, 2d_{\text{lo}}, t] &= \frac{1}{\sqrt{2}} \left(\text{tmp}[2f+1, d, t - T_0 - d] + \text{tmp}[2f+2, d, t - T_0] \right) \\ \text{out}[d_{\text{hi}}, 2d_{\text{lo}} + 1, t] &= \frac{1}{\sqrt{2}} \left(\text{tmp}[2f+1, d, t - T_1 - d - 1] + \text{tmp}[2f+2, d, t - T_1] \right) \end{aligned} \quad (1.19)$$

where on the RHS we define:³

$$d \equiv d_{\text{hi}}2^{l-1} + d_{\text{lo}} \quad T_0 = T_1 = d_{\text{hi}}2^{l-1}(2^{R-l+1} - 2f - 3) \quad (1.22)$$

(Warning: the quantities f, k, d_{lo} have slightly different meanings in cases 1 and 2!)

Note that subband dedispersion includes the full band (via $C_R = 1$), but the full-band output array is “reshaped” to $(2^{r-R}, 2^R, N_{\text{time}})$, whereas in the previous section it had shape $(2^r, N_{\text{time}})$.

1.4 Peak-finding

The final stage of the dedispersion transform is *peak-finding*. For now, we give a partial description – a future version of the notes will describe peak-finding in more detail. The input is the shape- $(2^{r-R}, M, N_{\text{time}})$ array from the previous section. The output is a shape- $(2^{r-R-K}, N_{\text{time}}/T_{\text{ds}})$ array, where T_{ds} is a time downsampling factor, and $K \geq 0$ allows additional DM downsampling. Conceptually, peak-finding is 3 steps:

- For each (d_{hi}, m) , the input array is a 1-d time series. We convolve the time series with P peak-finding kernels, described below, to obtain a shape $(2^{r-R}, M, N_{\text{time}}, P)$ array.
- We apply weights that depend on (d_{hi}, m, p) , to normalize the array so that each array element has variance 1. (Thus, array elements are normalized to “sigmas”.)
- We downsample the array to shape $(2^{r-R-K}, N_{\text{time}}/T_{\text{ds}})$. Downsampling is done with “max” rather than addition, so each array element corresponds to max detection significance over all downsampled parameters.

In implementation, these 3 steps are coalesced into one GPU kernel for speed (and also coalesced with the final stages of tree dedispersion), e.g. the shape $(2^{r-R}, M, N_{\text{time}}, P)$ array is never “materialized” in GPU memory.⁴

³In Eqs. (1.17), (1.22), we defined delays T, T_0, T_1 as follows:

$$T = d_{\text{hi}}2^l(2^{R-l} - 1 - f) \quad T_0 = T_1 = d_{\text{hi}}2^{l-1}(2^{R-l+1} - 2f - 3) \quad (1.20)$$

These definitions reflect what is currently implemented (in the GPU kernels and in **ReferenceTree**), but in hindsight these definitions may make more sense:

$$T = (d_{\text{hi}}2^l + d_{\text{lo}})(2^{R-l} - 1 - f) \quad T_0 = (d_{\text{hi}}2^{l-1} + d_{\text{lo}})(2^{R-l+1} - 2f - 3) \quad T_1 = T_0 + (2^{R-l} - f - 2) \quad (1.21)$$

We may switch the code to this convention in the future.

⁴Another comment on implementation. A **PeakFindingKernel** takes $D_{\text{ds}} = 2^{R+K}$ as the constructor argument **PeakFindingKernelParams::dm_downsampling**; it reads a $(2^{r-R}, M)$ array and reduces over the low K bits of the first index

1.4.1 Peak-finding kernels

There is always one profile ($p = 0$) consisting of a single unsmoothed sample. The remaining kernels are organized into $\Lambda \equiv \log_2 W_{\max}$ levels $0 \leq \lambda < \Lambda$,⁵ where $W_{\max}$ (a power of two) is the config parameter `max_kernel_width`, and each level contributes three profiles. The total number of profiles is therefore

$$P = 3 \log_2 W_{\max} + 1. \quad (1.23)$$

The edge case $W_{\max} = 1$ is allowed: there are then no levels, and the single sample $p = 0$ is the only profile. The building block at level λ is the *unnormalized running boxcar sum* of width 2^λ ,

$$b_\lambda[t] \equiv \sum_{i=0}^{2^\lambda-1} x[t-i], \quad (1.24)$$

computed by the recursion

$$b_0[t] = x[t], \quad b_{\lambda+1}[t] = b_\lambda[t] + b_\lambda[t - 2^\lambda]. \quad (1.25)$$

Note that (1.25) is a *plain sum*: there is no $1/\sqrt{2}$ in the boxcar downsampling. (All normalization is deferred to the weights.)

At each level λ we form up to four kernels, denoted $h_{\lambda,q}$ where $0 \leq q \leq 3$:

$$\begin{aligned} h_{\lambda,0} &= [\underbrace{1, \dots, 1}_{2^\lambda}] \\ h_{\lambda,1} &= [\underbrace{1, \dots, \dots, 1}_{2^{\lambda+1}}] \\ h_{\lambda,2} &= [\underbrace{\frac{1}{2}, \dots}_{2^\lambda}, \underbrace{1, \dots}_{2^\lambda}, \underbrace{\frac{1}{2}, \dots}_{2^\lambda}] \\ h_{\lambda,3} &= [\underbrace{\frac{1}{2}, \dots}_{2^\lambda}, \underbrace{1, \dots, \dots, 1}_{2^{\lambda+1}}, \underbrace{\frac{1}{2}, \dots}_{2^\lambda}] \end{aligned} \quad (1.26)$$

The profile index $0 \leq p < P$ is related to (λ, q) by $p = 3\lambda + q$, with the special value $p = 0$ for $(\lambda, q) = (0, 0)$. Thus level λ contributes $p = 3\lambda + 1, 3\lambda + 2, 3\lambda + 3$, and $p = 0$ (i.e. $h_{0,0} = [1]$, the single sample above) sits outside the levels. The single-boxcar kernel $h_{\lambda,0}$ never appears for $\lambda > 0$, since $h_{\lambda,0} = b_\lambda = h_{\lambda-1,1}$ would be redundant. So one may equally say that level 0 contributes $p = 0, 1, 2, 3$ and each higher level contributes three profiles; the two descriptions agree except at $W_{\max} = 1$, where there are no levels at all and $p = 0$ is the only profile.

1.5 Time-downsampled trees and early triggers

So far, we have described a single `DedispersionTree` which searches up to slope 1 (i.e. full-band dispersion delay equal to 2^r time samples), and “triggers” when the pulse arrives at the lowest frequency (highest tree-freq). It will be useful to define a larger `Dedisperser` object, consisting of multiple trees, as follows:

- **Downsampled trees** in order to search to higher DM. If we time-downsample the data by a factor 2^γ , and run rank- r dedispersion as usual, then we can search to a max dispersion delay which is larger by a factor 2^γ . When we time-downsample by a factor 2^γ , we add time samples and multiply by $2^{-\gamma/2}$, in order to leave the variance unchanged (assuming uncorrelated samples).

itself. A `CoalescedDdKernel12` instead fixes $2^{R+K} = 2^{\text{rank1}}$ from its own ranks, and asserts that the caller’s `dm_downsampling` agrees. See comments in code.

⁵We use λ and Λ here to avoid ambiguity with the subband level l in §1.3. In the code, the level index appears as `l` in `PfVarianceConvolver.peak_finding_kernels()` and as `lgw` in the kernel generator `PeakFinder`.

If $\gamma > 0$, then we only keep the upper half (in delay) of the dedispersion output, since the lower half was already searched by primary tree ($\gamma - 1$). Thus, each value of γ corresponds to a different DM range, ordered from low to high DM.

- **Early triggers** in order to reduce triggering latency, especially at high DM. By default, we only trigger when the pulse arrives at the highest tree-freq $f = 2^r$ (i.e. lowest radio frequency). If we run parallel tree dedispersion over the lowest 2^{r-e} tree-freqs (i.e. highest radio frequencies), then the triggering time will be smaller by a factor 2^e . This comes at the expense of losing low frequencies, so not all bursts will be detected in the early trigger.

The **Dedisperser** consists of multiple trees parameterized by (γ, e) . In the dedispersion config file, the syntax is (from `chord_sb2_et.yml`):

```

toplevel_tree_rank: 16    # rank r of "top-level" tree (gamma=0, e=0)

primary_trees:
- {num_early_triggers: 0, ...} # This line creates one tree with gamma=0, e=0
- {num_early_triggers: 1, ...} # This line creates two trees with gamma=1, e=0,1
- {num_early_triggers: 2, ...} # This line creates three trees with gamma=2, e=0,1,2
- {num_early_triggers: 3, ...} # This line creates four trees with gamma=3, e=0,1,2,3

```

In the code, data structures are as follows:

- There is one “top-level” tree with $\gamma = e = 0$ (no time downsampling, no early triggers) and rank $r = \text{toplevel_tree_rank}$.
- There are four “primary” trees with $e = 0$ (no early triggers) and $\gamma = 0, 1, 2, 3$. Each primary tree searches a different DM range, ordered from low to high DM.
- Each primary tree corresponds to multiple **DedispersionTrees** with $0 \leq e \leq \text{num_early_triggers}$. These trees (for fixed γ) search the same DM range, with different threshold frequencies for triggering, ordered from latest in time (lowest frequency) to earliest (highest frequency).

The integer γ is called the *primary tree index*, and the integer e is called the *early trigger level*. Conceptually, each **DedispersionTree** just runs “vanilla” rank- $(r - e)$ tree dedispersion on the first 2^{r-e} tree-freqs, downsampled in time by 2^γ , and then drops the lower DM-half of the outputs if $\gamma > 0$.⁶ For a more intuitive presentation of these trees, see Tab. 1 and Figs. 2, 3.

Finally, a comment on subbands. The subbands of §1.3 are declared once, in the config file, and apply to trees $(\gamma, 0)$ without early triggers (i.e. $e = 0$). For early trees ($e > 0$), we truncate the counts array $(C_0, \dots, C_R) \rightarrow (C_0, \dots, C_{R-e})$, then clamp each surviving C_l to its maximum allowed value (see Eq. (1.12)). Intuitively, early trees search all subbands which “fit” in the tree’s frequency range. A consequence of this rule: if early triggers are specified, then subbands must also be specified (more precisely, C_{R-e} must be > 0).

⁶This description suggests that each tree is an independent computation run “from scratch”. There are a lot of tricks in the code that reuse computations between trees, i.e. the **Dedisperser** is faster than running the constituent **DedispersionTrees** independently.

γ	e	Full-band delay range (s)	DM range (pc cm ⁻³)	trigger (MHz)	latency range (s)
0	0	0.0–65.5	0–1481	300	0.0–65.5
1	0	65.5–131.1	1481–2962	300	65.5–131.1
	1			416	32.8–65.5
2	0	131.1–262.1	2962–5924	300	131.1–262.1
	1			416	65.5–131.1
	2			567	32.8–65.5
3	0	262.1–524.3	5924–11847	300	262.1–524.3
	1			416	131.1–262.1
	2			567	65.5–131.1
	3			750	32.8–65.5

Table 1: The ten `DedispersionTrees` of the CHORD config `chord_sb2_et.yml` (rank $r = 16$; four primary trees). Each tree is identified by a (γ, e) pair. The DM range and *full-band* dispersion delay range depend only on γ . The trigger frequency depends only on e , and the latency range is $2^{-e} \times (\text{delay range})$. (Generated with `pirate_frb show dedisperser -v configs/dedispersion/chord_sb2_et.yml`.)

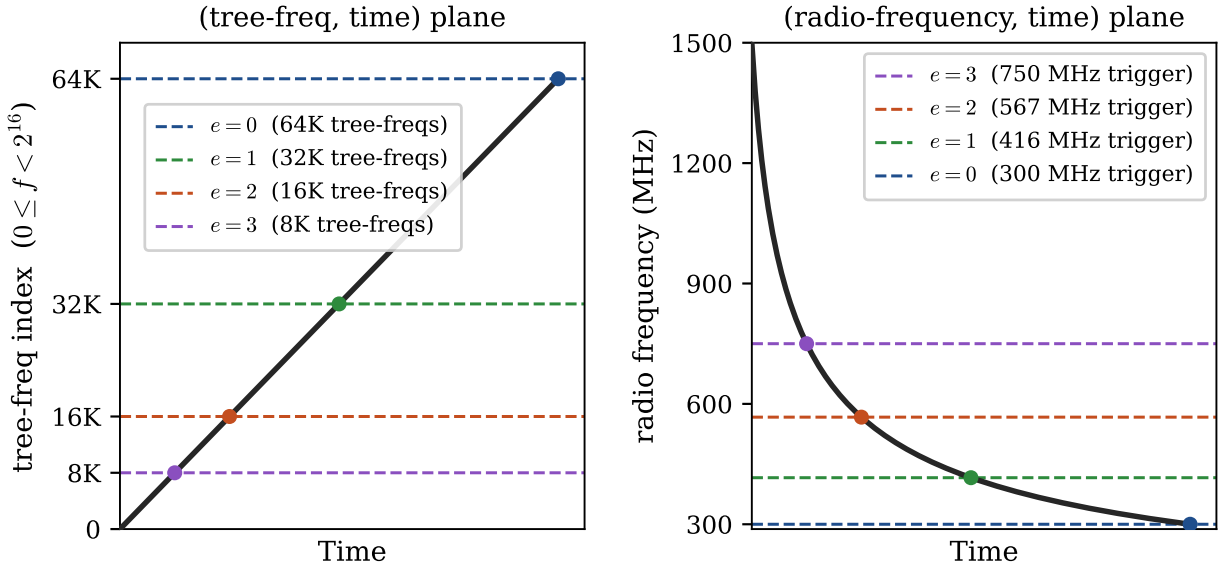


Figure 2: A visualization of early triggers, in radio frequency (right panel) or the “tree-freq” reparametrization used in the code (left panel). Trees with early triggers search the lower part of the tree-freq range, i.e. the high part of the radio frequency range. A tree with $e > 0$ triggers earlier (by a factor 2^e) than the main tree. The plot assumes CHORD params (300–1500 MHz, $r = 16$, $0 \leq e \leq 3$).

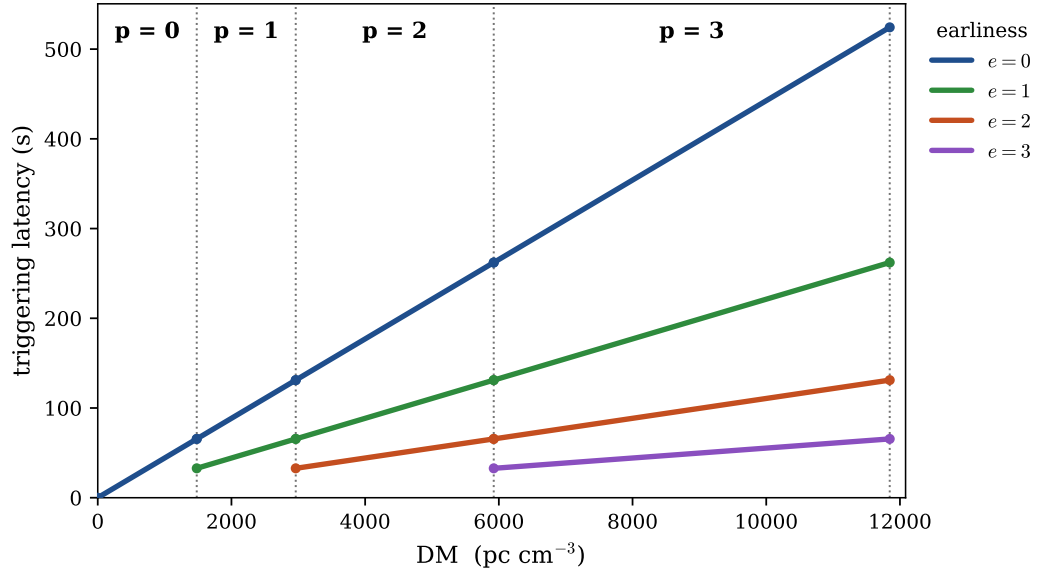


Figure 3: The ten `DedispersionTrees` from Table 1, shown as line segments in the (DM, triggering latency) plane. For each tree (γ, e) , the value of γ determines the DM range, and the value of e determines the slope.

1.6 Dedispersion output arrays

The dedispersion output consists of a pair of 2-d arrays (`out_max`, `out_argmax`) for each beam and `DedispersionTree`, indexed by (`dm`, `time`). The purpose of this section is to describe these arrays in a little more detail.

1.6.1 Parameter definitions

The pirate `DedispersionPlan` defines the following global (i.e. not per-tree) parameters:

$$\begin{aligned} T_{\text{in}} &= \text{nt_in} && \text{(input time samples per time chunk)} \\ r_{\text{top}} &= \text{toplevel_tree_rank} && \text{(toplevel tree rank, specified in the config file)} \end{aligned}$$

and the following per-tree parameters:

$$\begin{aligned} \gamma &= \text{primary_tree_index} && \text{(primary tree index; input is time-downsampled by } 2^\gamma) \\ e &= \text{early_trigger_level} && \text{("earliness" of trigger)} \\ T_{\text{ds}} &= \text{time_downsampling} && \text{(time coarse-graining factor in peak-finding kernel)} \\ D_{\text{ds}} &= \text{dm_downsampling} && \text{(dm coarse-graining factor, denoted } 2^{R+K} \text{ in §1.4)} \end{aligned}$$

These parameters can be inspected with `pirate_frb show dedisperser -v`, and are also passed from pirate to the grouper via the `dedispersion_plan` yaml metadata.

Warning. Unfortunately, it's not currently possible to choose the coarse-graining factors ($T_{\text{ds}}, D_{\text{ds}}$) in the config file! The dedisperser will "choose them for you", and they need not be the same in all trees. If your downstream code depends on how they're chosen, you should put asserts in your code, in case the values change in the future. This is something I hope to improve soon!

1.6.2 Output array shapes

The `out_max` and `out_argmax` arrays (for a fixed tree and beam), have the same shape (`ndm_out`, `nt_out`), given by:

$$\text{ndm_out} = \begin{cases} 2^{r_{\text{top}}-e}/D_{\text{ds}} & \text{if } \gamma = 0 \\ 2^{r_{\text{top}}-e-1}/D_{\text{ds}} & \text{if } \gamma \geq 1 \end{cases} \quad (1.27)$$

$$\text{nt_out} = \frac{T_{\text{in}}}{2^\gamma T_{\text{ds}}} \quad (1.28)$$

The *full-band* delay range searched by each tree (expressed as an integer number samples, before any per-tree time downsampling has been done) is:

$$d_{\text{lo}} = \begin{cases} 0 & \text{if } \gamma = 0 \\ 2^{r_{\text{top}}+\gamma-1} & \text{if } \gamma \geq 1 \end{cases} \quad d_{\text{hi}} = 2^{r_{\text{top}}+\gamma} \quad (1.29)$$

To explain where Eqs. (1.27)–(1.29) come from, we walk through steps in the code:

- On the time axis, we start with a length- T_{in} input array, downsample by a factor 2^γ , run tree dedispersion, and downsample the output (max-ing, not averaging!) by T_{ds} . This implies Eq. (1.28).
- The DM axis is a bit more complicated. First consider the base tree (no downsampling or early triggers), and assume $D_{\text{ds}} = 1$ for simplicity. We have:

$$d_{\text{lo}} = 0 \quad d_{\text{hi}} = 2^{r_{\text{top}}} \quad \text{ndm_out} = 2^{r_{\text{top}}} \quad (1.30)$$

Then we include additional complications one at a time. If we downsample the input by 2^γ , then d_{hi} gets multiplied by 2^γ . Recall that for $\gamma > 0$, we also discard the lower half of the output DMs (which was already searched by primary tree ($\gamma - 1$)). Therefore:

$$d_{\text{lo}} = \begin{cases} 0 & \text{if } \gamma = 0 \\ 2^{r_{\text{top}} + \gamma - 1} & \text{if } \gamma > 0 \end{cases} \quad d_{\text{hi}} = 2^{r_{\text{top}} + \gamma} \quad \text{ndm_out} = \begin{cases} 2^{r_{\text{top}}} & \text{if } \gamma = 0 \\ 2^{r_{\text{top}} - 1} & \text{if } \gamma > 0 \end{cases} \quad (1.31)$$

If this is an early-triggered tree with $e > 0$, then the *full-band* delay range is unchanged, but the spacing between trial DMs increases by 2^e , so **ndm_out** decreases by the same factor:

$$d_{\text{lo}} = \begin{cases} 0 & \text{if } \gamma = 0 \\ 2^{r_{\text{top}} + \gamma - 1} & \text{if } \gamma > 0 \end{cases} \quad d_{\text{hi}} = 2^{r_{\text{top}} + \gamma} \quad \text{ndm_out} = \begin{cases} 2^{r_{\text{top}} - e} & \text{if } \gamma = 0 \\ 2^{r_{\text{top}} - e - 1} & \text{if } \gamma > 0 \end{cases} \quad (1.32)$$

Finally, we downsample the output (max-ing, not averaging!) by D_{ds} . This decreases **ndm_out** by a factor D_{ds} , leaving d_{lo} , d_{hi} unchanged. This implies Eqs. (1.27), (1.29).

1.6.3 The out_argmax array

Each element of the **out_max** array is a “coarse-grained” snr value, defined by taking a maximum over a certain set of parameters: frequency subbands, pulse widths, fine-grained DMs, and fine-grained arrival times. The **out_argmax** array consists of integer “tokens” which encode the “winning” parameters, that are responsible for the max snr.

Before writing down the token encoding, recall from §1.4, §1.6 that the DM downsampling factor is of the form $D_{\text{ds}} = 2^{R+K}$. The first R bits are part of the “multiplet” $0 \leq m < M$ (see Eq. (1.13)), and we introduce a new index $0 \leq \mu < 2^K$ for the remaining K bits.

In this notation, the token encoding is:

$$(32\text{-bit } \text{argmax_token}) = (t) \mid (p \ll 8) \mid (m \ll 16) \mid (\mu \ll 24) \quad (1.33)$$

i.e. one byte per field, where $0 \leq t < T_{\text{ds}}$ is a fine-grained arrival time, $0 \leq p < P$ is a peak-finding profile index (encoding pulse width and profile shape), $0 \leq m < M \leq 114$ is a “multiplet” (i.e. (frequency subband, fine-grained DM) pair), and $0 \leq \mu < 2^K \leq 16$ is part of the fine-grained DM as defined in the previous paragraph. Note that m and μ occupy separate bytes, so decoding a token does not require knowing K . The arrival time t has a small p -dependent time shift applied (see code for details).

In practice, you shouldn’t need to know the details of the token encoding – there are helper functions that are intended to hide it. In particular, the method **FrbGrouper.create_events()** decodes tokens to recover the “winning” parameters (min/max freq, DM, timestamp, width) for each **out_max** array value.

When using **FrbGrouper.create_events()**, note that the output timestamp is defined to be the arrival time at the lowest frequency of the band (400 MHz in CHIME, or 300 MHz in CHORD), even if the FRB is detected in a subband. If there are early triggers, then the timestamp can be **significantly later** ($\sim 10^2$ seconds) than the end of the time-chunk. With or without early triggers, the timestamp can also be **slightly earlier** than the beginning of the time-chunk, due to the finite width of the peak-finding kernels.

2 Implementation

2.1 Non-recursive formulation of tree dedispersion

Recall from §1.2 that tree dedispersion $\text{DD}(r)$ is a linear operator whose inputs and outputs are both $(2^r, N_{\text{time}})$, defined recursively via Eqs. (1.5), (1.8). Here, we show that $\text{DD}(r)$ has the following non-recursive definition, where we sum over frequency channels with lags:

$$\text{out}[d, t] = \frac{1}{2^{r/2}} \sum_{f=0}^{2^r-1} \text{in}[f, t - L_r(f, d)] \quad (2.1)$$

The “lag” function $L_r(f, d)$ is defined by:

$$L_r(f, d) = \sum_{i=0}^{r-1} \sum_{j=0}^{r-1} \Omega_{i+j-r} (1 - f_i) d_j \quad (2.2)$$

where Ω_n is defined by:

$$\Omega_n \equiv \begin{cases} 2^n & \text{if } n \geq 0 \\ 1 & \text{if } n = -1 \\ 0 & \text{if } n < -1 \end{cases} \quad (2.3)$$

and $f_j, d_j \in \{0, 1\}$ are the bits in the base-2 representation:

$$f = [f_{r-1} \cdots f_1 f_0]_2 \quad d = [d_{r-1} \cdots d_1 d_0]_2 \quad (2.4)$$

In Appendix A.1, we show that the nonrecursive definition (2.1) of tree dedispersion is equivalent to our previous recursive definition in Eqs. (1.5), (1.8).

2.2 The splitting identity

The purpose of this section is to explain the following “splitting identity”: if $r = r' + r''$, then

$$\text{DD}(r) = (\text{DD}(r'') \otimes 1) \circ \text{LAG}[(2^{r''} - 1 - f'') d'] \circ (1 \otimes \text{DD}(r')) \quad (2.5)$$

where the notation will take a little time to explain.

We split the frequency index $0 \leq f < 2^r$ and DM-index $0 \leq d < 2^r$ into low/high parts as follows:

$$\begin{aligned} f &= 2^{r'} f'' + f' & \text{where } 0 \leq f' < 2^{r'} \text{ and } 0 \leq f'' < 2^{r''} \\ d &= 2^{r''} d' + d'' & \text{where } 0 \leq d' < 2^{r'} \text{ and } 0 \leq d'' < 2^{r''} \end{aligned} \quad (2.6)$$

Using these splittings, we can view tree dedispersion $\text{DD}(r)$ as a linear operator with input/output shapes $(2^{r''}, 2^{r'}, N_{\text{time}})$. In the splitting identity (2.5), the operators $(\text{DD}(r'') \otimes 1)$ and $(1 \otimes \text{DD}(r'))$ are naturally defined with this array shape. The middle operator $\text{LAG}[(2^{r''} - 1 - f'') d']$ is defined as follows, on the same array shape $(2^{r''}, 2^{r'}, N_{\text{time}})$.

$$\text{out}[f'', d', t] = \text{in}[f'', d', t - (2^{r''} - 1 - f'') d'] \quad (2.7)$$

Note the asymmetry in the splittings of f, d in Eq. (2.6): the “primed” index f' is the “fine” frequency index, whereas the “primed” index d' is the “coarse” DM-index. This asymmetry arises from the details of the tree dedispersion algorithm.

One consequence: in implementation, we sometimes end up using bit-reversed DM indices so that array

computations are “in-place”, rather than via temporary arrays. (Note that the same phenomenon happens for the FFT algorithm!) We’ll gloss over this detail in the notes, where our goal is to explain the math but not necessarily the details of the implementation.

When contemplating the splitting identity (2.5), the following diagram may help visualize the RHS:

$$(f, t) \xrightarrow{\text{Reshape}} (f'', f', t) \xrightarrow{1 \otimes \text{DD}(r')} (f'', d', t) \xrightarrow{\text{LAG}} (f'', d', t) \xrightarrow{\text{DD}(r'') \otimes 1} (d'', d', t) \xrightarrow{\text{Transpose+Reshape}} (d, t) \quad (2.8)$$

where tuples such as (f'', d', t) indicate whether an input/output array is 2-d or 3-d, and indicate whether each index is interpreted as a frequency or a DM.

We defer the proof of the splitting identity (2.5) to Appendix A.2.

2.3 GPU implementation

In this section, we describe our fast GPU implementation of tree dedispersion. This section is slightly simplified compared to the full implementation, but accurately captures the “vibe”.

Our goal is to implement $(1_M \otimes \text{DD}(r))$, where 1_M is the identity operator over a spectator index $0 \leq m < M$. In the text below, we usually streamline notation by omitting the spectator index. The data is a real-time stream which arrives in “chunks” with shape $(M, 2^r, N_{\text{time}})$. The kernel looks like:

```
// Length-M and length-2^r axes have caller-specified strides, time axis has stride 1.
__global__ void dd(float *arr, long mstride, long dstride, float *persistent_state);
```

The kernel operates in-place (i.e. no separate in and out args). The “persistent” state is a kernel-defined ring buffer that is carried over from one time chunk to the next (see below). (Context: we’re assuming that as real-time data arrives, the kernel is launched for $0 \leq t < N_{\text{time}}$, then launched for $N_{\text{time}} \leq t < 2N_{\text{time}}$, etc.)

For small r , we can implement tree dedispersion in a single warp, and launch the kernel with M warps. First consider $r = 1$. From the perspective of a single warp, $\text{DD}(1)$ has input/output shape $(2, N_{\text{time}})$ and is defined by:

$$\begin{aligned} \text{out}[0, t] &= \frac{1}{\sqrt{2}} (\text{in}[0, t] + \text{in}[1, t]) \\ \text{out}[1, t] &= \frac{1}{\sqrt{2}} (\text{in}[0, t-1] + \text{in}[1, t]) \end{aligned} \quad (2.9)$$

This is straightforward to implement in a single warp, but note that we need to save the value of $\text{in}[0, N_{\text{time}} - 1]$ for the next time chunk. Thus, for $r = 1$ the persistent state consists of one element per warp (M elements per kernel).

Next consider a single-warp implementation with $r > 1$. We use the splitting identity (2.5) to write:

$$\text{DD}(r) = \underbrace{(\text{DD}(1) \otimes 1_{2^{r-1}})}_{\equiv \mathcal{D}_2} \circ \text{LAG}[(1 - f'') d'] \circ \underbrace{(1_2 \otimes \text{DD}(r-1))}_{\equiv \mathcal{D}_1} \quad (2.10)$$

Suppose by induction that we have a single-warp implementation of $\text{DD}(1)$ and $\text{DD}(r-1)$. The $\text{LAG}[\dots]$ operator just means that we put the outputs of $\mathcal{D}_1$ into a logical ring buffer, before feeding them to $\mathcal{D}_2$. Here, “logical” means that the ring buffer is held in registers of a single warp. This is persistent state: we save the ring buffer to global memory at the end of the kernel, and restore it at the beginning of the next kernel.

As r increases, the single-warp implementation eventually becomes impractical. A little thought shows

that the persistent-state size (in elements, not bytes) P_r satisfies the recursion:

$$P_r = \underbrace{\binom{2^{r-1}}{\mathcal{D}_2}}_{\mathcal{D}_2} + \underbrace{\left(\sum_{d'=0}^{2^{r-1}-1} d'\right)}_{\text{LAG}} + \underbrace{(2P_{r-1})}_{\mathcal{D}_1} \quad (2.11)$$

with initial condition $P_1 = 1$. The solution to this recursion relation is:

$$P_r = 2^{r-2}(2^r + r - 1) \quad (2.12)$$

Additionally, our implementation uses 2^r registers/thread to store a length-32 sliding time window of array values. Currently, we use the single-warp implementation for $r \leq 4$.

To explain the $r > 4$ case, consider $r = 8$ for concreteness. We apply the splitting identity (2.5) to write:

$$\text{DD}(8) = \underbrace{(\text{DD}(4) \otimes 1_{16})}_{\equiv \mathcal{D}_2} \circ \text{LAG}[(15 - f'') d'] \circ \underbrace{(1_{16} \otimes \text{DD}(4))}_{\equiv \mathcal{D}_1} \quad (2.13)$$

The operators $(\text{DD}(4) \otimes 1_{16})$ and $(1_{16} \otimes \text{DD}(4))$ can be distributed across 16 warps in a single threadblock (using our single-warp implementation of $\text{DD}(4)$ described above). In between, we write the data to a ring buffer in **GPU shared memory**, in order to apply lags and in order to move data between different warps. This idea can be applied for any r , to obtain a single-threadblock implementation with one shared memory read/write cycle.

As r increases, the single-threadblock implementation eventually becomes impractical, due to register and shared memory limits. The total persistent state size P_r (now interpreted as elements per threadblock) is still given by Eq. (2.12)⁷, but note that the underlying ring buffers are split between registers and shared memory. Currently, we use the single-threadblock implementation for $5 \leq r \leq 8$.

To explain the $r > 8$ case, consider $r = 16$ for concreteness. We use the splitting identity (2.5) to write:

$$\text{DD}(16) = \underbrace{(\text{DD}(8) \otimes 1_{256})}_{\equiv \mathcal{D}_2} \circ \text{LAG}[(255 - f'') d'] \circ \underbrace{(1_{256} \otimes \text{DD}(8))}_{\equiv \mathcal{D}_1} \quad (2.15)$$

The operators $(\text{DD}(8) \otimes 1_{256})$ and $(1_{256} \otimes \text{DD}(8))$ can be distributed across 256 threadblocks in a single kernel (using our single-threadblock implementation of $\text{DD}(8)$ described above).

The middle operator $\text{LAG}[\dots]$ in Eq. (2.15) is really a no-op. It just means that we put the outputs of $\mathcal{D}_1$ in a ring buffer in GPU global memory, before reading them with $\mathcal{D}_2$. Thus, Eq. (2.15) tells us how to divide the $\text{DD}(16)$ operation into two GPU kernels, separated by a ring buffer in **GPU global memory**. We use this “two-pass” implementation of $\text{DD}(r)$ for $9 \leq r \leq 16$. (The current code only implements $r \leq 16$, but if larger r were needed, then a “three-pass” implementation would cover $17 \leq r \leq 24$.)

In the above implementation, the main idea is to use the splitting identity recursively, to split tree dedispersion across multiple kernels, then parallelize over threadblocks in a kernel, then parallelize over threads in a warp.

⁷This is not obvious, but follows from the following identity: if $r = r' + r''$, then

$$P_r = 2^{r'} P_{r''} + 2^{r''} P_{r'} + \sum_{d'=0}^{2^{r'}-1} \sum_{f''=0}^{2^{r''}-1} (2^{r''} - 1 - f'') d' \quad (2.14)$$

2.4 Extracting subbands from a split dedispersion [Claude]

In §1.3.2, we defined subbanded dedispersion via a lot of temporary arrays. In our GPU kernels, we use coalescing tricks to avoid writing these temporary arrays to global GPU memory (the temporaries exist only in registers). In this section, we work out the details.

§1.3.2 states the construction for a monolithic DD(r). This section says what changes when DD(r) is split as $r = r_0 + r_1$ (Eq. 2.5, whose r', r'' these are), and is meant to orient a reader before the kernel code.

Where the subbands are. A level- l subband is the array after $r - R + l$ steps DDS in case 1, or one step earlier plus an explicit merge across the subband midpoint in case 2. The first stage performs steps $0, \dots, r_0 - 1$, so every subband lands in the second stage iff $R \leq r_1$, which `DedispersionConfig::validate()` enforces (its “Check 2”). Equivalently: put a DD(R) at the outside of the second stage, and the level- l subbands are its intermediates, level 0 being its input and the full band its output. Being intermediates of a single stage is what keeps them in registers.

What lag it needs. The extraction needs

$$\text{LAG}[(2^R - 1 - F) d_{\text{hi}}] \quad (2.16)$$

where $0 \leq F < 2^R$ is the coarse-freq index (the f of case 1 at $l = 0$) and d_{hi} is the output coarse delay. Given that lag, the DD(R) which follows produces the T 's of cases 1 and 2 by itself, with no further lag logic.

Let $K \equiv r_1 - R$. If $K = 0$ the lag above is exactly the LAG of the $r_0 + r_1$ split, which the implementation applies anyway (a ring buffer in shared memory), so the subband lags cost nothing. If $K > 0$ then $d_{\text{hi}} = 2^K d' + \mu$, where d' is the first stage's delay and $0 \leq \mu < 2^K$ comes from an inner DD(K), and the lag splits as

$$(2^R - 1 - F) d_{\text{hi}} = \underbrace{2^K (2^R - 1 - F) d'}_{\text{already applied by the } r_0 + r_1 \text{ split}} + \underbrace{(2^R - 1 - F) \mu}_{\text{one constant per } (F, \mu)} \quad (2.17)$$

So $K > 0$ costs one compile-time-constant lag per second-stage register, and buys 2^K values of d_{hi} per unit of d' .

Names in the code. r_1 is `dd_rank1` = $\lceil \text{dd_rank}/2 \rceil$, and K is the count of “extra DM” bits folded into the argmax token. The pipeline carries it only through `dm_downsampling` = 2^{R+K} (`DedispersionTree` and `PeakFindingKernelParams`); `ReferencePeakFindingKernel::K` derives it. The kernel is emitted by `cuda_generator/SbDedisperser.py` and registered by `GpuSbDedispersionKernel` (`include/pirate/DedispersionKernel`). `ReferenceTree` is the CPU reference.

2.5 Alignment constraints and the MegaRingbuf

In the previous section, we stated (just after Eq. (2.15)) that the $\text{LAG}[\dots]$ operator in global memory is a no-op. This is a slight simplification: in implementation, we constrain global memory lags to be 128-byte aligned. (The reason why is a long story – see the last few paragraphs in this section!)

Let E be the “segment size”, or the number of data elements per 128-byte cache line:

$$E \equiv \frac{128}{\text{sizeof}(\text{dtype})} = \begin{cases} 32 & \text{for a float32 kernel} \\ 64 & \text{for a float16 kernel} \end{cases} \quad (2.18)$$

Taking DD(16) as an example, we split the lag $\lambda(f'', d') = (255 - f'') d'$ appearing in Eq. (2.15) into “aligned” and “residual” parts:

$$\lambda(f'', d') \equiv (255 - f'') d' \quad \lambda_a(f'', d') \equiv E \left\lfloor \frac{\lambda(f'', d')}{E} \right\rfloor \quad \lambda_r(f'', d') \equiv \lambda(f'', d') - \lambda_a(f'', d') \quad (2.19)$$

We factorize DD(16) as follows (compare with Eq. (2.15)):

$$\text{DD}(16) = \underbrace{(\text{DD}(8) \otimes 1_{256}) \circ \text{LAG}[\lambda_r(f'', d'')]}_{\equiv \mathcal{D}_2} \circ \text{LAG}[\lambda_a(f'', d'')] \circ \underbrace{(1_{256} \otimes \text{DD}(8))}_{\equiv \mathcal{D}_1} \quad (2.20)$$

The residual lag $\lambda_r(f'', d'')$ is coalesced into the second-stage dedispersion kernel $\mathcal{D}_2$ (slightly increasing the size of its persistent state), and the aligned lag $\lambda_a(f'', d'')$ is a no-op in global memory. The dedispersion kernels have a boolean compile-time parameter `apply_input_residual_lags` which indicates whether the lag $\lambda_r(f'', d'')$ is applied before dedispersion. (There is a related parameter `input_is_downsampled_tree` which will be explained in the next section.)

As an aside, something analogous happens internally in the float16 dedispersion kernels for $5 \leq r \leq 8$. In this case, recall from the previous section that we use a single-threadblock, multi-warp kernel with a lagged ring buffer in shared memory. In order to keep the shared memory IO 32-bit aligned, we split the 16-bit lag into an even (aligned) part, and a $\{0, 1\}$ -valued residual part. The shared memory ring buffer applies the aligned part, and the residual part is coalesced into the second half of the dedispersion.

Finally, we explain why we constrain global memory lags to be 128-byte aligned. An obvious reason is that it allows global memory access to be cache-aligned, but this has a small performance impact. The larger reason is as follows.

So far, we’ve stated that intermediate arrays between dedispersion kernels are stored in a GPU global memory ring buffer. This is a simplification: we actually store intermediate arrays in the `MegaRingbuf`, a complicated data structure which lives partially in GPU memory, and partially in host memory. This is because the total ring buffer size is much larger than GPU memory. The `MegaRingbuf` is responsible for organizing ring buffer state so that “short-lag” elements of the ring buffer stay on the GPU, and “long-lag” elements spill to host memory (in order to minimize PCIe bandwidth). The `MegaRingbuf` requires 128-byte aligned lags, in order to ensure that all host-GPU copies are 128-byte aligned. Here, the alignment really matters – profiling shows that host-gpu bandwidth is significantly reduced if addresses are not 128-byte aligned (but does not benefit from 256-byte alignment).

2.6 Downsampled trees and the LaggedDownsampler

As mentioned previously (§1.5), for downsampled trees, we only keep the upper half (in delay d) of the dedispersion transform output. In this section, we explain some tricks to make this more efficient in implementation, by not computing the lower half, and coalescing kernels.

First we introduce notation for the operator that we want to implement:

$$U \circ \text{DD}(r) \circ \text{TDS}(\gamma) \quad (2.21)$$

The operator U is the “upper half” operator whose input is a shape $(2^r, N_{\text{time}}/2^\gamma)$ array and whose output is a shape $(2^{r-1}, N_{\text{time}}/2^\gamma)$ array, defined by `out[d, t] = in[2r-1 + d, t]`. The operator $\text{TDS}(\gamma)$ downsamples in time by a factor 2^γ (with a factor $2^{-\gamma/2}$), i.e. it maps a shape $(2^r, N_{\text{time}})$ array to a shape $(2^r, N_{\text{time}}/2^\gamma)$ array.

Now consider the following identity (non-obvious, proved in Appendix A.3):

$$U \circ \text{DD}(r) = \text{DD}(r-1) \circ \text{LAG}[2^{r-1} - 1 - f] \circ \text{LS} \quad (2.22)$$

Here, LS is a “lagged sum” operator defined as follows. The input is a shape $(2^r, N_{\text{time}}/2^\gamma)$ array, and the output is a shape $(2^{r-1}, N_{\text{time}}/2^\gamma)$ array, defined by:

$$\text{out}[f, t] = \frac{1}{\sqrt{2}} (\text{in}[2f, t-1] + \text{in}[2f+1, t]) \quad (2.23)$$

In implementation, the RHS of Eq. (2.22) can be more efficient than the LHS, since the RHS avoids computing

(and throwing away) the lower half of the dedispersion output (i.e. the dedispersion is rank $(r - 1)$ instead of rank r).

To compute $\text{DD}(r - 1)$ on the GPU (for the large r -values that arise for CHIME and CHORD), we apply the splitting identity (2.5) to $\text{DD}(r - 1)$. We choose a splitting (note $r - 1$ in the leftmost equation):

$$r - 1 = r' + r'' \quad f = 2^{r'} f'' + f' \quad d = 2^{r''} d' + d'' \quad (2.24)$$

and write:

$$\text{DD}(r - 1) = (\text{DD}(r'') \otimes 1) \circ \text{LAG}[(2^{r''} - 1 - f'') d'] \circ (1 \otimes \text{DD}(r')) \quad (2.25)$$

$$\text{LAG}[2^{r-1} - 1 - f] = \text{LAG}[2^{r'}(2^{r''} - 1 - f'')] \circ \text{LAG}[2^{r'} - 1 - f'] \quad (2.26)$$

In the last line, we have split the lag into two pieces which depend only on f'' and f' . Putting the two equations together, we get:

$$\text{DD}(r - 1) \circ \text{LAG}[2^{r-1} - 1 - f] = (\text{DD}(r'') \otimes 1) \circ \text{LAG}[(2^{r''} - 1 - f'')(d' + 2^{r'})] \circ (1 \otimes \text{DD}(r')) \circ \text{LAG}[2^{r'} - 1 - f'] \quad (2.27)$$

where we used the fact that the operators $\text{LAG}[2^{r'}(2^{r''} - 1 - f'')]$ and $(1 \otimes \text{DD}(r'))$ commute, since the lag depends on f'' but not f' . Next, we split the leftmost lag on the RHS into aligned and residual parts (as in Eq. (2.19)):

$$\lambda(f'', d') \equiv (2^{r''} - 1 - f'')(d' + 2^{r'}) \quad \lambda_a(f'', d') \equiv E \left[\frac{\lambda(f'', d')}{E} \right] \quad \lambda_r(f'', d') \equiv \lambda(f'', d') - \lambda_a(f'', d') \quad (2.28)$$

Putting all of this together:

$$\begin{aligned} U \circ \text{DD}(r) \circ \text{TDS}(\gamma) &= \underbrace{(\text{DD}(r'') \otimes 1) \circ \text{LAG}[\lambda_r(f'', d')]}_{\equiv \mathcal{D}_2} \circ \text{LAG}[\lambda_a(f'', d')] \circ \underbrace{(1 \otimes \text{DD}(r'))}_{\equiv \mathcal{D}_1} \\ &\quad \circ \underbrace{\text{LAG}[2^{r'} - 1 - f'] \circ \text{LS} \circ \text{TDS}(\gamma)}_{\equiv \text{LDS}(r', \gamma)} \end{aligned} \quad (2.29)$$

where the last line defines the “lagged downsampling kernel” $\text{LDS}(r', \gamma)$. As in §2.5, we coalesce the residual lag $\text{LAG}[\lambda_r(f'', d')]$ into the second stage dedispersion kernel $\mathcal{D}_2$. Note however that the form of $\lambda_r(f'', d')$ is different from before (Eqs. (2.19), (2.28) differ by $d' \rightarrow (d' + 2^{r'})$). The GPU kernel has a boolean argument `input_is_downsampled_tree` to select which residual lag is applied.

We have now written the downsampled tree $U \circ \text{DD}(r) \circ \text{TDS}(\gamma)$ as the composition of three kernels LDS , $\mathcal{D}_1$, $\mathcal{D}_2$. The “aligned” lag operator $\text{LAG}[\lambda_a(f'', d')]$ is implemented as a no-op, using the `MegaRingbuf` (§2.5).

For large downsampling factor γ , the most expensive step in this length-three chain is the relatively trivial kernel $\text{LDS}(r', \gamma)$, for memory bandwidth reasons:

- $\text{LDS}(r', \gamma)$ has input shape $(2^r, N_{\text{time}})$ and output shape $(2^{r-1}, N_{\text{time}}/2^\gamma)$.
- $\mathcal{D}_1$ and $\mathcal{D}_2$ have input/output shapes $(2^{r-1}, N_{\text{time}}/2^\gamma)$.

The memory bandwidth requirements are driven by the $\text{LDS}(r', \gamma)$ input array, which is *the same for all* γ . Therefore, our “production” `LaggedDownsamplingKernel` is a coalesced kernel which computes $\text{LDS}(r', \gamma)$ for all values of γ simultaneously, i.e. the kernel has one input array and γ_{max} output arrays. This coalesced kernel improves performance significantly.

2.7 Putting it all together: the GPU kernel diagram

In this section, we put everything together, by describing the exact sequence of GPU kernels in the dedisperser. We’ll also describe a few implementation details that we didn’t mention yet (early trigger implementation, coalesced kernels). When reading the bullet points below, it may be helpful to refer in parallel to Figure 4, which shows the GPU kernel chain visually.

- Recall (§2.2–2.6, Eqs. (2.20), (2.29)) that we implement tree dedispersion (at CHIME/CHORD scale) as two kernels, separated by the **MegaRingbuf**. We’ll refer to these kernels as “stage-1” and “stage-2” kernels.
- Recall (§1.5) that some trees may have early triggers, in addition to the “main” trigger. In implementation, all of the triggers for a tree share a stage-1 kernel, but have different stage-2 kernels that run at different times. This crucial detail makes early triggers less expensive than might initially be expected. It complicates the implementation of the **MegaRingbuf**, since each segment in the ring buffer has one producer and multiple consumers (with different lags).

In general, there is a many-to-one mapping between stage-2 kernels and stage-1 kernels. We have one stage-1 kernel per primary tree (terminology from §1.5), and one stage-2 kernel per **DedispersionTree**.

- Recall (§1.4, §1.6) that the output of each stage-2 dedispersion kernel is the input to a peak-finding kernel. In implementation, we optimize by defining a **CoalescedDdKernel2**, which coalesces the stage-2 dedispersion kernel and the peak-finding kernel.
- The first GPU kernel in Figure 4 is the **TreeGriddingKernel** (denoted TKG) which takes a $(N_{\text{freq}}, N_{\text{time}})$ array to a $(2^n, N_{\text{time}})$ array, as described in §1.1.
- Next, two kernels operate in parallel: the stage-1 dedispersion kernel (denoted DD1) for the base (i.e. non-downsampled) tree, and the **LaggedDownsamplingKernel** (LDS), which prepares input for the downsampled (i.e. $\gamma \geq 1$) trees.
- The LDS kernel has γ_{max} output arrays (one for each downsampled tree). Each of these is the input for a distinct DD1 kernel.
- The DD1 kernels write their output to the **MegaRingbuf**, and each **CoalescedDdKernel2** (denoted CDD2 in Figure 4) reads its input from the **MegaRingbuf**. Each CDD2 kernel writes dedispersion output arrays, in the format described in §1.6.

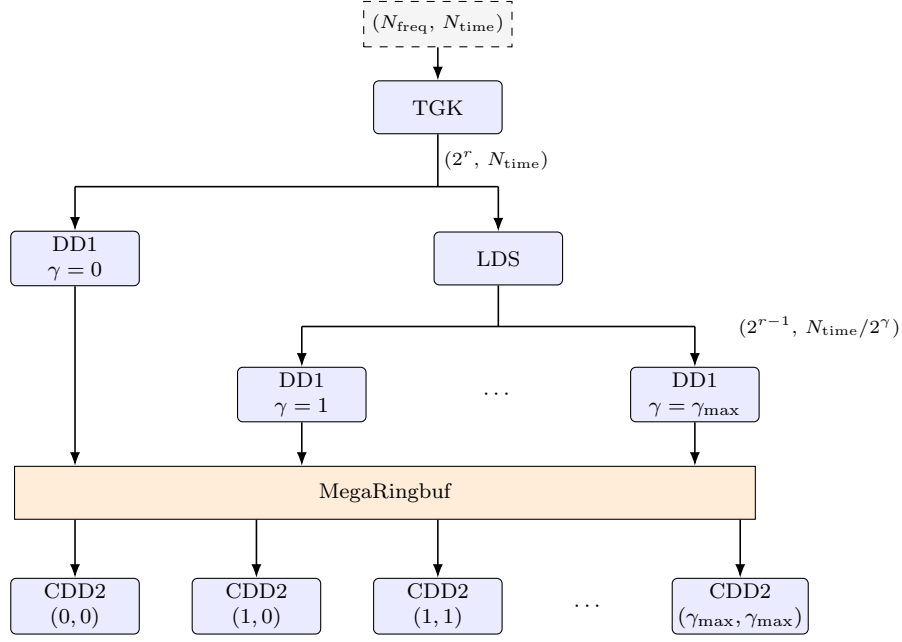


Figure 4: The GPU kernels of the dedisperser, and the arrays which connect them. The labels `TGK`, `LDS`, `DD1`, `CDD2` stand for `TreeGriddingKernel`, `LaggedDownsamplingKernel`, `GpuDedispersionKernel` (stage 1), and `CoalescedDdKernel12`. There is one `DD1` kernel per primary tree (or downsampling level γ), and one `CDD2` kernel per `DedispersionTree` (or (γ, e) pair, where e is an early trigger level). Operations internal to the `MegaRingbuf` are not shown (but nontrivial – they copy data between the GPU and host, and synchronize with the GPU kernels). See text in §2.7 for more info.

A Derivations [Claude]

A.1 The non-recursive formulation of tree dedispersion

In this appendix we prove that the recursive definition of tree dedispersion (Eqs. 1.5, 1.8) is equivalent to the non-recursive definition (Eq. 2.1), with lag function $L_r(f, d)$ given by Eq. (2.2). The proof is by induction on r , and uses two structural observations.

Observation 1: block structure. By inspection of its defining equations, the dedispersion step $\text{DDS}(k)$ computes output coarse-freq block f using only input blocks $2f$ and $2f + 1$, via formulas which do not depend on f or r . Unrolling this over the composition (1.8): after steps $\text{DDS}(0), \dots, \text{DDS}(k-1)$, coarse-freq block f of the intermediate array depends only on tree-freq channels $2^k f \leq (\text{tree-freq}) < 2^k(f+1)$, and equals rank- k dedispersion $\text{DD}(k)$ applied to those 2^k channels. In particular, isolating the last step of a rank- $(r+1)$ composition,

$$\text{DD}(r+1) = \text{DDS}(r) \circ (1 \otimes \text{DD}(r)) \quad (\text{A.1})$$

where we view the rank- $(r+1)$ input array as having shape $(2, 2^r, N_{\text{time}})$, and $(1 \otimes \text{DD}(r))$ applies $\text{DD}(r)$ independently to the lower and upper half-bands (same tensor notation as in the splitting identity, Eq. 2.5).

Observation 2: delay bits. By the defining equations of $\text{DDS}(k)$, the output delay is $D = 2\delta + e$, where δ is the input delay and $e \in \{0, 1\}$ selects the even/odd output line. Thus each step appends one new low bit to the delay index; at the final step, $D_0 = e$ and $D_j = \delta_{j-1}$ for $j \geq 1$.

Base case. For $r = 0$, the composition (1.8) is empty, so $\text{DD}(0)$ is the identity; and $L_0(0, 0) = 0$ is an empty sum, so Eq. (2.1) reduces to $\text{out}[0, t] = \text{in}[0, t]$. (The reader preferring a nontrivial base case may check $r = 1$: $L_1(f, d) = \Omega_{-1}(1 - f_0)d_0 = (1 - f_0)d_0$, so Eq. (2.1) reads $\text{out}[d, t] = 2^{-1/2}(\text{in}[0, t-d] + \text{in}[1, t])$, which matches $\text{DDS}(0)$ applied to a $(2, 1, N_{\text{time}})$ input.)

Inductive step. Assume Eq. (2.1) holds at rank r . By Eq. (A.1) and the defining equations of $\text{DDS}(r)$ (with coarse output index $f = 0$, input delay δ , and output delay $D = 2\delta + e$):

$$\text{out}[2\delta + e, t] = \frac{1}{\sqrt{2}} \left(\text{lo}[\delta, t - \delta - e] + \text{hi}[\delta, t] \right) \quad e \in \{0, 1\} \quad (\text{A.2})$$

where lo, hi denote $\text{DD}(r)$ applied to channels $[0, 2^r)$ and $[2^r, 2^{r+1})$ respectively (the lag $\delta + e$ on the lower block combines the $t - d$ and $t - d - 1$ lines). Substituting the inductive hypothesis,

$$\text{out}[D, t] = \frac{1}{2^{(r+1)/2}} \left(\sum_{g=0}^{2^r-1} \text{in}[g, t - \delta - e - L_r(g, \delta)] + \sum_{g=0}^{2^r-1} \text{in}[2^r + g, t - L_r(g, \delta)] \right) \quad (\text{A.3})$$

Comparing with Eq. (2.1) at rank $r+1$, it suffices to prove two identities for the lag function:

$$L_{r+1}(2^r + g, D) = L_r(g, \delta) \quad (\text{A.4})$$

$$L_{r+1}(g, D) = L_r(g, \delta) + \delta + e \quad (\text{A.5})$$

where (by Observation 2) the bits of D are $D_0 = e$ and $D_j = \delta_{j-1}$ for $j \geq 1$.

To prove Eq. (A.4), write $F = 2^r + g$, so that $F_r = 1$ and $F_i = g_i$ for $i < r$. In the double sum

$$L_{r+1}(F, D) = \sum_{i=0}^r \sum_{j=0}^r \Omega_{i+j-(r+1)} (1 - F_i) D_j \quad (\text{A.6})$$

the $i = r$ terms vanish, since $(1 - F_r) = 0$. For $i \leq r-1$, the $j = 0$ term contains $\Omega_{i-(r+1)}$ with index $i - (r+1) \leq -2$, hence also vanishes. Substituting $j = j' + 1$ in the remaining terms gives $\Omega_{i+j'-r}(1 - g_i)\delta_{j'}$ summed over $0 \leq i, j' \leq r-1$, which is $L_r(g, \delta)$.

To prove Eq. (A.5), take $F = g$, so that $F_r = 0$. The $i \leq r-1$ terms give $L_r(g, \delta)$ exactly as before, and

the $i = r$ terms contribute

$$(1 - F_r) \sum_{j=0}^r \Omega_{j-1} D_j = \Omega_{-1} D_0 + \sum_{j=1}^r 2^{j-1} D_j = e + \delta \quad (\text{A.7})$$

This completes the induction.

We remark that $\delta + e = \lceil D/2 \rceil$, so Eq. (A.5) says that the lower half-band lags the upper half-band by half of the total delay, rounded up. The case $\Omega_{-1} = 1$ in the definition of Ω_n is precisely this round-up – the algebraic counterpart of the $(t - d - 1)$ in the odd-delay line of DDS(k).

A.2 The splitting identity

Next we prove the splitting identity (2.5), taking the non-recursive definition (2.1) as the starting point. The key statement is an “additivity” identity for the lag function: with the index splittings of Eq. (2.6),

$$L_r(f, d) = L_{r'}(f', d') + (2^{r''} - 1 - f'') d' + L_{r''}(f'', d'') \quad (\text{A.8})$$

Reduction to Eq. (A.8). Expand the right-hand side of the splitting identity (2.5) using the non-recursive form (2.1) at ranks r' and r'' . Acting on an input array $\text{in}[f'', f', t]$, the three stages give:

$$\begin{aligned} (1 \otimes \text{DD}(r')) : \quad & A[f'', d', t] = \frac{1}{2^{r'/2}} \sum_{f'} \text{in}[f'', f', t - L_{r'}(f', d')] \\ \text{LAG} : \quad & B[f'', d', t] = A[f'', d', t - (2^{r''} - 1 - f'') d'] \\ (\text{DD}(r'') \otimes 1) : \quad & C[d'', d', t] = \frac{1}{2^{r''/2}} \sum_{f''} B[f'', d', t - L_{r''}(f'', d'')] \end{aligned} \quad (\text{A.9})$$

Since each stage is a sum of lagged copies of its input, the lags add under composition:

$$C[d'', d', t] = \frac{1}{2^{r/2}} \sum_{f'', f'} \text{in}[f'', f', t - L_{r'}(f', d') - (2^{r''} - 1 - f'') d' - L_{r''}(f'', d'')] \quad (\text{A.10})$$

using $2^{-r'/2} 2^{-r''/2} = 2^{-r/2}$. Comparing with $\text{DD}(r)$ in the form (2.1), the splitting identity holds if and only if the total lag above equals $L_r(f, d)$ for every (f, d) – i.e. if and only if Eq. (A.8) holds.

Proof of Eq. (A.8). Under the splittings (2.6), the bits decompose as $f_i = f'_i$ for $0 \leq i < r'$ and $f_{r'+i''} = f''_{i''}$ for $0 \leq i'' < r''$; likewise $d_j = d'_j$ for $0 \leq j < r''$ and $d_{r''+j'} = d'_{j'}$ for $0 \leq j' < r'$. Now split the double sum (2.2) into four blocks:

- **(fine freq) $\times$ (low DM):** $i < r'$ and $j < r''$. Here $i + j - r \leq (r' - 1) + (r'' - 1) - r = -2$, so $\Omega_{i+j-r} = 0$ and the block vanishes: fine-frequency bits never couple to sub-block DM bits.
- **(fine freq) $\times$ (high DM):** $i < r'$ and $j = r'' + j'$. Here $\Omega_{i+j-r} = \Omega_{i+j'-r'}$, so the block is $\sum_{i < r'} \sum_{j' < r'} \Omega_{i+j'-r'} (1 - f'_i) d'_{j'} = L_{r'}(f', d')$.
- **(coarse freq) $\times$ (low DM):** $i = r' + i''$ and $j < r''$. Here $\Omega_{i+j-r} = \Omega_{i''+j-r''}$, giving $L_{r''}(f'', d'')$ by the same reindexing.
- **(coarse freq) $\times$ (high DM):** $i = r' + i''$ and $j = r'' + j'$. Here $i + j - r = i'' + j' \geq 0$, so Ω is in its 2^n regime and the block factorizes:

$$\left(\sum_{i''=0}^{r''-1} 2^{i''} (1 - f''_{i''}) \right) \left(\sum_{j'=0}^{r'-1} 2^{j'} d'_{j'} \right) = (2^{r''} - 1 - f'') d' \quad (\text{A.11})$$

which is the LAG term.

Summing the four blocks proves Eq. (A.8), and hence the splitting identity.

We remark that each of the three cases in the definition of Ω_n plays a distinct role here: the 2^n regime produces the factorized cross term (the linear extrapolation of the coarse-DM slope d' across the $(2^{r''}-1-f'')$ blocks above block f''); the shift invariance of Ω_{i+j-r} under $(i, j, r) \rightarrow (i+a, j+b, r+a+b)$ makes the two diagonal blocks reproduce the lower-rank lag functions; and the cutoff $\Omega_n = 0$ for $n \leq -2$ expresses the fact that fine-frequency structure never couples to sub-block DM bits. Compare also the time lags T appearing in the subbanded-dedispersion construction (§1.3), which have the same form as the cross term.

A.3 The upper-half identity

Finally we prove the upper-half identity (2.22), which is what lets a downsampled tree – where only the upper half of the DM range is kept – be computed at rank $(r-1)$ instead of rank r . It is a corollary of the splitting identity, so no new computation is needed: it is the $d' = 1$ slice of Eq. (2.5) at $r' = 1$.

Take $r' = 1$ and $r'' = r-1$ in the splittings (2.6). The frequency splitting $f = 2^{r'} f'' + f'$ becomes

$$f = 2f'' + f' \quad 0 \leq f'' < 2^{r-1}, \quad f' \in \{0, 1\} \quad (\text{A.12})$$

so f'' indexes a *pair* of adjacent tree-freq channels and f' selects a channel within the pair. The DM splitting $d = 2^{r''} d' + d''$ becomes

$$d = 2^{r-1} d' + d'' \quad d' \in \{0, 1\}, \quad 0 \leq d'' < 2^{r-1} \quad (\text{A.13})$$

so d' is simply the *top bit* of the DM index. Therefore the upper half of the DM range is exactly the $d' = 1$ slice:

$$d \geq 2^{r-1} \iff d' = 1, \quad \text{and then} \quad d = 2^{r-1} + d'' \quad (\text{A.14})$$

i.e. U selects $d' = 1$, and the surviving DM index of $U \circ \text{DD}(r)$ is d'' .

With these values, the splitting identity (2.5) reads

$$\text{DD}(r) = (\text{DD}(r-1) \otimes 1_2) \circ \text{LAG}[(2^{r-1} - 1 - f'') d'] \circ (1_{2^{r-1}} \otimes \text{DD}(1)) \quad (\text{A.15})$$

on arrays of shape $(2^{r-1}, 2, N_{\text{time}})$. Now restrict each of the three factors to $d' = 1$, working right to left:

- **The DD(1) factor becomes LS.** The operator $(1 \otimes \text{DD}(1))$ applies DD(1) along the length-2 axis, taking $(f'', f', t) \rightarrow (f'', d', t)$. By the definition (2.9) of DD(1), its $d' = 1$ output is

$$\text{out}[f'', 1, t] = \frac{1}{\sqrt{2}} (\text{in}[f'', 0, t-1] + \text{in}[f'', 1, t]) \quad (\text{A.16})$$

Since $\text{in}[f'', 0, \cdot]$ and $\text{in}[f'', 1, \cdot]$ are the tree-freq channels $2f''$ and $2f''+1$, this is precisely the definition (2.23) of LS. (The $d' = 0$ output is the *unlagged* sum of the pair, which is what the discarded lower DM-half is built from.)

- **The LAG factor becomes $\text{LAG}[2^{r-1} - 1 - f'']$.** Substituting $d' = 1$ into the lag $(2^{r-1} - 1 - f'') d'$ leaves $(2^{r-1} - 1 - f'')$, a per-channel lag on the length- 2^{r-1} axis. This is the middle factor of (2.22), with f'' playing the role of f .
- **The DD($r-1$) factor is unchanged.** The operator $(\text{DD}(r-1) \otimes 1_2)$ dedisperses along the f'' axis with the d' axis a spectator, so its restriction to the $d' = 1$ slice is just DD($r-1$), taking $f'' \rightarrow d''$.

Composing the three restricted factors gives Eq. (2.22). As a check on the normalization, LS contributes $2^{-1/2}$ and DD($r-1$) contributes $2^{-(r-1)/2}$, for a total of $2^{-r/2}$, matching DD(r).

The identity at the level of lag functions. It is worth recording what (2.22) says about L_r , since this is the form in which it is easiest to check. Writing $f = 2g + b$ with $b \in \{0, 1\}$, the additivity identity (A.8) at $r' = 1$, $d' = 1$, $d'' = \tilde{d}$ gives

$$L_r(2g + b, 2^{r-1} + \tilde{d}) = L_{r-1}(g, \tilde{d}) + (2^{r-1} - 1 - g) + (1 - b) \quad (\text{A.17})$$

where we used $L_1(b, 1) = \Omega_{-1}(1 - b) = 1 - b$. Read right to left, the three terms are exactly the three stages of (2.22): the LS lag of one sample applied to the even channel of each pair and none to the odd channel, the LAG term, and the rank- $(r - 1)$ tree. Note that $(1 - b)$ carries no $\tilde{d}$ -dependence – the lowest frequency bit does not couple to any DM bit below the top one, by the cutoff $\Omega_n = 0$ for $n \leq -2$. That is what makes the construction possible: within a pair of adjacent tree-freqs, *every* DM in the upper half sees the same one-sample offset, so the pair can be collapsed by LS before the DM index is resolved.